

School Management System

Core Java Project Assignment

Project Objective

Your task: Build a console-based School Management System using Core Java. The project should demonstrate OOP, interfaces, exception handling, arrays, a custom dynamic array, menu-driven programming and object serialization.

Important: Do NOT use Java Collection Framework classes such as `ArrayList` , `LinkedList` , `HashMap` , etc.

Do NOT use the Java Stream API or Lambda expressions.

You must create and manage the dynamic array yourself.

1. Student Class

Create a class named `Student` .

Create approximately 5–6 fields. For example:

Field	Data Type
studentId	int
name	String
age	int
gender	String
course	String
marks	double

The Student class must contain:

- Private fields
- No-argument constructor
- Parameterized constructor
- Getters

- Setters
- `toString()`
- `Serializable`

2. School Class

Create a class named `School` .

Create 4–5 school-related fields.

Example:

```
private int schoolId;
private String schoolName;
private String address;
private String principalName;
private Student[] students;
private int studentCount;
```

The most important requirement is that the `School` class must maintain a **Student array**.

3. Create Your Own Dynamic Array

This is the most important part of the project.

You are NOT allowed to use `ArrayList` or any other collection. You must make the array dynamic yourself.

Initially, create a small array:

```
Student[] students = new Student[5];
```

Suppose all five positions become occupied and another student needs to be inserted.

Your program must:

1. Create a new, larger `Student` array.
2. Normally double the capacity.
3. Copy the old students into the new array.
4. Replace the old array with the new array.
5. Insert the new student.

Example:

```
Old Array:

[ S1 ][ S2 ][ S3 ][ S4 ][ S5 ]
    capacity = 5
```

New Array:

```
[ S1 ][ S2 ][ S3 ][ S4 ][ S5 ][   ][   ][   ][   ][   ]  
  
capacity = 10
```

You must understand and implement this logic yourself.

4. ISchool Interface

Create an interface:

```
public interface ISchool {  
  
    void insert(Student student) throws Exception;  
  
    void update(int studentId, Student student) throws Exception;  
  
    void delete(int studentId) throws Exception;  
  
    Student view(int studentId) throws Exception;  
}
```

These methods represent the basic operations of the School Management System.

5. SchoolService

Create `SchoolService` and implement `ISchool`.

You must implement all four operations:

Operation	Required Logic
Insert	Check duplicate student ID, increase array capacity if required, and insert the student.
Update	Search the student by ID and update the student's information.
Delete	Find the student, remove it and shift the remaining elements to the left.
View	Search by student ID and display/return the student.

Reusable Search Logic

Do not write the same search loop again and again. Create one reusable method.

```
private int findStudentIndex(int studentId) {  
  
    for (int i = 0; i < studentCount; i++) {  
  
        if (students[i].getStudentId() == studentId) {  
            return i;  
        }  
    }  
}
```

```
}  
  
    return -1;  
}
```

Delete Logic

If the array contains:

```
[ S1 ][ S2 ][ S3 ][ S4 ][ S5 ]
```

Suppose **S3** is deleted.

The remaining elements must shift left:

Before:

```
[ S1 ][ S2 ][ S3 ][ S4 ][ S5 ]
```

After:

```
[ S1 ][ S2 ][ S4 ][ S5 ][ null ]
```

Then decrease `studentCount` .

6. School Menu

Create a menu-driven application using:

- `do-while`
- `switch-case`
- `Scanner`

```
=====
                SCHOOL MANAGEMENT SYSTEM
=====

1. Insert Student
2. Update Student
3. Delete Student
4. View Student
5. View All Students
6. View School Details
7. Save Data
8. Exit
```

Enter your choice:

Menu Rules

- Menu must repeat until the user selects Exit.
- Use `switch-case` for menu operations.
- Do not put business logic directly inside the menu.
- Call `SchoolService` methods from the menu.
- Handle invalid choices properly.
- Handle invalid user input without crashing the program.

7. Object Serialization

Your application must permanently store the School object using object serialization.

Use:

- `ObjectOutputStream`
- `ObjectInputStream`
- `FileOutputStream`
- `FileInputStream`

Use a file such as:

```
school.dat
```

Saving

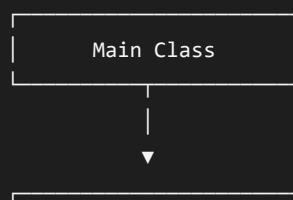
When data changes, save the School object using `ObjectOutputStream`.

Loading

When the application starts, use `ObjectInputStream` to read the previously saved School object.

Important: Student, School, and every object that is part of the serialized object graph must be Serializable.

8. Complete Project Flow





9. Exception Handling

Handle situations such as:

- Student ID does not exist.
- Duplicate Student ID.
- Invalid student details.
- Invalid menu choice.
- Invalid numeric input.
- File does not exist.
- Serialization/deserialization failure.

You may create custom exceptions such as:

```
StudentNotFoundException  
DuplicateStudentException
```

10. Strict Restrictions

Do NOT use:

- ArrayList
- LinkedList
- Vector
- HashMap
- HashSet
- Any Java Collection Framework class
- Stream API
- Lambda expressions
- Database
- Spring / Spring Boot
- External frameworks

The purpose of this project is to implement the storage logic ourselves using arrays.

11. Test Cases

#	Test	Expected Result
1	Insert student	Student is successfully inserted.
2	Insert duplicate ID	Duplicate student is rejected.
3	Fill array completely	Array automatically grows.
4	Update student	Student information is updated.
5	Update invalid ID	Student-not-found message.
6	Delete first student	Remaining students shift correctly.
7	Delete middle student	Remaining students shift correctly.
8	Delete last student	Last occupied position becomes null.
9	View student	Correct student details are displayed.
10	Restart application	Previously saved data is loaded.

12. Submission Requirements

- ☐ Student.java
- ☐ School.java
- ☐ ISchool.java
- ☐ SchoolService.java
- ☐ SchoolMenu.java
- ☐ Main.java

- ☐ Custom exception classes
- ☐ Serialization / file-storage class
- ☐ Complete working project
- ☐ README with project explanation

13. Final Goal

By completing this project, you should be able to explain:

- How objects are created and managed.
- How encapsulation works.
- How an interface defines operations.
- How a service class implements business logic.
- How a normal Java array can be converted into a dynamic array.
- How elements are shifted during deletion.
- How duplicate and missing IDs are handled.
- How do-while and switch-case create a menu-driven application.
- How ObjectOutputStream saves objects.
- How ObjectInputStream restores objects.

You must be able to explain your code during project review.